HACK APERTUS 2026 · ONLINE HACK · TRACK 2B, OWN PROJECT

Ari Widget Translator
Technical report

Team LiveMap: Norbert Kremer, Claudio Romano, Ewan Jones, Tomas Witek, Luboš Matejčík

Model swiss-ai/Apertus-v1.5-8B and 70B, self-hosted

Repository github.com/nkr-livemap/hackapertus-widget-translator

Demo track_2b/docs/ari-apertus-demo.mp4 (100 s)

1. Summary

Ari (myari.app) is a city companion app for Zurich built from small, self-contained widgets. Each widget ships its user-facing copy as a typed string catalog in English and German. Switzerland has four national languages, and the widgets had none of the others. The Widget Translator closes that gap: a team member picks a widget and a target language (French or Italian), a workflow translates the catalog, runs deterministic checks (placeholders, ICU plural branches, widget markers, Swiss spelling) and an LLM review, the team member corrects single keys while two live preview frames show the widget in English and in the new language, and the result lands as a code pull request with the new `fr` or `it` block. The product tool was designed and built during the hackathon window (plan 15.10.2026, merged 16.10.2026, 9 catalogs, 1 002 keys). This repository adds the Apertus path: a standalone runner with the same prompt and check logic that talks to Apertus over an OpenAI-compatible endpoint and is what `make run` executes.

2. Architecture

The product tool is part of the Ari monorepo (`livemapai/frontend`, pull request #3432) and has three parts:

Part	Where it runs	What it does
`packages/widget-runtime`	in every widget	`fr` and `it` as message locales with English fallback; message templates (small ICU subset) so function-valued messages cross a JSON boundary; extraction artifact of all catalogs; emitter that writes a Biome-clean locale block back into `strings.ts`
`apps/agents`	agents host (Mastra)	workflow `widgetTranslation`: translate in chunks of 12 keys, deterministic checks, LLM review, finalize; staff-only route
`apps/ari-landing`	myari.app	translator page: catalog and language choice, key-by-key editing, drafts shared across the team, live preview via `?messageLocale=`, download, or a draft PR from local development

The standalone runner in `src/translate.mjs` reproduces the translate and check steps without the monorepo: it reads `data/catalogs.json` (the extraction artifact of all 9 catalogs), sends one system message plus one user message per chunk to the model, parses the structured JSON answer, runs the checks and writes `out/<catalog>.<locale>.json` with a status per key (OK, warning, error, uncertain).

Target architecture (mandatory)

a) On-premise. Inference runs on the team's own GPU server (Dell XE7745, 8× NVIDIA H200 NVL, data centre Stollen, Lucerne) with vLLM behind a LiteLLM proxy. Apertus v1.5 8B and 70B have been served there since 27.09.2026 and have carried the inference for this hackathon's participants. The runner needs only `LLM_BASE_URL`, `LLM_NAME` and `LLM_API_KEY`, so the same image also works against **c) a sovereign Swiss cloud** (Infomaniak serves `swiss-ai/Apertus-v1.5-70B` through the same OpenAI-compatible API).

External dependencies: build time, the `node:22-alpine` image; runtime, the inference endpoint only. No other service is called.

---

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (default), `swiss-ai/Apertus-v1.5-70B` by setting `LLM_NAME`
- **How it is used:** inference with structured JSON output
- **Where it runs:** self-hosted vLLM on the team's GPU server (`https://hackapertus.livemap.sh/v1`), alternatively Infomaniak

Prompt design follows a constraint we hit earlier in the hackathon window: Apertus v1.5 rejects a second system message (HTTP 400 "Invalid message role"), while our agents send two (instructions plus working memory). The translation prompt therefore uses exactly one system message with the rules (keep `{placeholder}` names, keep ICU plural and select syntax with an `other` branch, keep the `|underline|` and `[link]` markers, `ss` never `ß`, informal register as in the German reference) and one user message with the chunk of keys including the German reference translation. The German reference matters: it fixes register and length, which the English source leaves open. Temperature 0.2 for translation and 0 for the review, 90 s timeout per call, one retry on malformed JSON in the product workflow.

In the product, the model is set with `WIDGET_TRANSLATION_MODEL`; the Apertus endpoint is selected by pointing it at the LiteLLM rack. The default worker chain of the product remains the team's standard chain, so Apertus is an opt-in there.

---

4. Data

`data/catalogs.json` (163 KB) holds the English and German source strings of 9 widget catalogs, 1 002 keys, extracted from the Ari source code by the team's own tooling. It is the team's own copy, visible to every user of myari.app, and is published here under CC-BY-4.0. No personal data is involved. Translations are generated at run time and written to `out/`, which is not committed.

Catalog	Keys
components/strings	20
widgets/ari/strings	123
widgets/directions/strings	10
widgets/feedback-user-research/strings	52
widgets/made-in-zurich/strings	62
widgets/new-places-to-eat-and-drink/strings	145
widgets/welcome/strings	5
widgets/whats-on/strings	452
widgets/zurich-galleries/strings	133

5. Evaluation

Success for a UI translation is binary per key: it must compile and render without breaking the widget (placeholders, plurals, markers), and a native speaker must accept it. The deterministic checks measure the first part exactly, the LLM review and the `uncertain` flag triage the second for human review.

Setup	Metric	Result
Product workflow, general worker model, `welcome` to French, 5 keys	keys passing all checks	5 of 5, about 26 s end to end
Product workflow, general worker model, 25 template keys from `zurich-galleries` and `whats-on` to French, one request	status per key	21 OK, 3 warnings, 1 error, 32 s; placeholders, plural branches and markers intact in all 25
Product workflow, general worker model, `zurich-galleries` to French and Italian, full catalog	keys translated; status per key	129 of 129 in each language. French: 108 OK, 16 warnings, 5 errors; Italian: 116 OK, 12 warnings, 1 error
Same run, human spot check	errors the review caught	French agreement errors, for example «23 ouvert maintenant» to «ouvertes», flagged with a fix
Same run, human spot check	errors the review missed	Italian «Nessuna vernissage» (should be «Nessun») in 3 keys, all marked OK
Standalone runner, Apertus 8B	keys passing all checks per catalog	not measured before the deadline, see Limitations

The last two rows are the reason native-speaker review stays a human step: the checks and the review catch broken structure and most grammar, but a fluent-sounding gender error in three keys passed both.

The runner prints the summary line and writes the per-key file so judges can reproduce the numbers for any catalog and locale with `make run CATALOG=widgets/whats-on/strings LOCALE=it`.

The demo video walks through the product flow as configured for Apertus; its screen recording was made with the general worker model, the same runs as in the table above.

6. Limitations

- The Apertus runs were not measured before the submission deadline. The product tool was verified end to end with a general-purpose worker model; the runner carries the same prompt and checks to Apertus, but its pass rates are open. Without `LLM_API_KEY` the runner falls back to a dry run that executes the checks against the German reference, so `make run` always completes.
- Only catalogs with the standard `strings.ts` shape are covered: 9 catalogs, that is 8 of the 50 widgets plus the shared rating component. Hard-coded copy in roughly 37 widgets and shared catalogs in other shapes are out of scope.
- 7 of 129 function-valued messages do not convert to templates and stay English.
- The live preview does not yet apply drafts to widgets that load through the frame delivery path (`welcome`); the standalone widget frame shows them correctly.
- Native-speaker review stays a human step. The tool flags uncertain keys, it does not replace the reviewer (see the Italian example in section 5).
- Swiss German is not a target yet, although it is the obvious next language for a Zurich app and the reason this project fits Hack Apertus.

7. Reproducibility

```
make run                                    # welcome -> fr, dry run without a key
LLM_API_KEY=... make run CATALOG=widgets/zurich-galleries/strings LOCALE=it
```

Env: `LLM_NAME` (default `swiss-ai/Apertus-v1.5-8B`), `LLM_BASE_URL` (default `https://hackapertus.livemap.sh/v1`), `LLM_API_KEY`. Docker, Node 22 inside the image, no GPU needed on the judge's machine. Temperature 0.2; outputs are not bit-reproducible across runs. Product code: `livemapai/frontend`, pull request #3432, merged 16.10.2026 07:52 UTC.

8. Next steps

- Measure Apertus 8B and 70B against the checks on all 9 catalogs and compare with the current worker model; switch the product default where Apertus holds.
- Swiss German as a target language: dialect output needs a register decision (which dialect, which spelling convention) and a native-speaker panel; the check layer and preview already work for any locale.
- Extend extraction to hard-coded copy so all widgets become translatable.

License

Source code in this repository (`src/`, `Makefile`, `Dockerfile`): Apache License 2.0. Report, documentation and video: Creative Commons Attribution 4.0 (CC-BY-4.0). `data/catalogs.json`: CDLA-Permissive-2.0. All HackApertus projects are open-sourced; the product monorepo referenced in this report is not part of the submission.

References

- Ari, https://myari.app
- Pull request «feat: widget translator», https://github.com/livemapai/frontend/pull/3432 (private repository)
- Apertus v1.5, https://huggingface.co/swiss-ai/Apertus-v1.5-8B and https://huggingface.co/swiss-ai/Apertus-v1.5-70B